

BS1.5

Plan

wir wollen die Konfiguration von Windows Servern mit Ansible umsetzen.

Dazu setzen wir eine Testumgebung aus einer Linux Ansible Control Node und zwei Windows-Servern um.

Control-Node

interface	ip	lan segment
NAT-Interface	DHCP	-
ens37	192.168.1.1/24	WAnsible

- Konfiguration
 - Control Node mit Netplan konfigurieren

```
# Let NetworkManager manage all devices on this system
network:
  version: 2
  renderer: NetworkManager
  ethernets:
    ens37:
      dhcp4: false
      addresses:
        - 192.168.1.1/24
```

WS1-Ansible (Managed Node)

interface	ip	lan segment
NAT-Interface	DHCP	-
Ethernet1	192.168.1.2/24	WAnsible

Konfiguration

Skript:

```
# ----- Script-Anfang -----
# Set device name
Rename-Computer -NewName 'WS1-Ansible' -Force
# Rename network adapter
Rename-NetAdapter -Name 'Ethernet0' -NewName 'Internet'

Rename-NetAdapter -Name 'Ethernet1' -NewName 'WAnsible'
# Setup dynamic for IPv4
Set-NetIPInterface -InterfaceAlias 'WAnsible' -AddressFamily IPv4 -Dhcp Disabled
```

```

New-NetIPAddress -InterfaceAlias 'WAnsible'-AddressFamily IPv4 -IPAddress
'192.168.1.2' -PrefixLength 24 -DefaultGateway "192.168.1.2"
# DNS-Client
Set-DnsClientServerAddress -InterfaceAlias 'WAnsible' -ResetServerAddresses
Set-DnsClientServerAddress -InterfaceAlias 'WAnsible' -ServerAddresses
("192.168.1.2")

# Set timezone and time (timeserver)
w32tm /config /syncfromflags:manual /manualpeerlist:"pool.ntp.org" /reliable:yes
/update
tzutil /s "W. Europe Standard Time"
# Allow pings in both directions
Set-NetFirewallRule -Name "CoreNet-Diag-ICMP4-EchoRequest-In" -Enabled True
Set-NetFirewallRule -Name "CoreNet-Diag-ICMP4-EchoRequest-Out" -Enabled True

Set-NetFirewallRule -Name "CoreNet-Diag-ICMP6-EchoRequest-In" -Enabled True
Set-NetFirewallRule -Name "CoreNet-Diag-ICMP6-EchoRequest-Out" -Enabled True
# RDP aktivieren
winRM quickconfig
Enable-PSRemoting -Force
Set-ItemProperty -Path 'HKLM:\System\CurrentControlSet\Control\Terminal Server' -
name "fDenyTSConnections" -value 0
# Reboot
Restart-Computer
# ----- Script-Ende-----

```

Zum DC heraufstufen (corp.com)

WS2-Ansible (Managed Node)

interface	ip	lan segment
NAT-Interface	DHCP	-
Ethernet1	192.168.1.3/24	WAnsible

Konfiguration

Skript

```

# ----- Script-Anfang -----
# Set device name
Rename-Computer -NewName 'WS2-Ansible' -Force
# Rename network adapter
Rename-NetAdapter -Name 'Ethernet0' -NewName 'Internet'

Rename-NetAdapter -Name 'Ethernet1' -NewName 'WAnsible'
# Setup dynamic for IPv4
Set-NetIPInterface -InterfaceAlias 'WAnsible' -AddressFamily IPv4 -Dhcp Disabled
New-NetIPAddress -InterfaceAlias 'WAnsible'-AddressFamily IPv4 -IPAddress
'192.168.1.3' -PrefixLength 24 -DefaultGateway "192.168.1.2"

```

```
# DNS-Client
Set-DnsClientServerAddress -InterfaceAlias 'WAnsible' -ResetServerAddresses
Set-DnsClientServerAddress -InterfaceAlias 'WAnsible' -ServerAddresses
("192.168.1.2")

# Set timezone and time (timeserver)
w32tm /config /syncfromflags:manual /manualpeerlist:"pool.ntp.org" /reliable:yes
/update
tzutil /s "W. Europe Standard Time"
# Allow pings in both directions
Set-NetFirewallRule -Name "CoreNet-Diag-ICMP4-EchoRequest-In" -Enabled True
Set-NetFirewallRule -Name "CoreNet-Diag-ICMP4-EchoRequest-Out" -Enabled True

Set-NetFirewallRule -Name "CoreNet-Diag-ICMP6-EchoRequest-In" -Enabled True
Set-NetFirewallRule -Name "CoreNet-Diag-ICMP6-EchoRequest-Out" -Enabled True
# RDP aktivieren
winRM quickconfig
Enable-PSRemoting -Force
Set-ItemProperty -Path 'HKLM:\System\CurrentControlSet\Control\Terminal Server' -
name "fDenyTSConnections" -value 0
# Reboot
Restart-Computer
# ----- Script-Ende-----
```

Domain beitreten und zum DC heraufstufen (corp.com)

- Ansible Konfiguration

Linux Control Node

- Ansible installieren:

```
sudo apt update
sudo apt install ansible
```

- Die Datei /etc/ansible/hosts bearbeiten

```
sudo nano /etc/ansible/hosts
```

Windows Managed Nodes

- WICHTIG!: Falls noch nicht durchs Skript geschehen Windows Remote Management aktivieren (Server Manager->Lokaler Server->Remotemanagement) oder

```
winRM quickconfig
Enable-PSRemoting -Force
```

Über HTTP verbinden (zertifikatlos und unverschlüsselt)

- Auf den Windows hosts unencrypted access und eingehend http zulassen:

```
Set-Item -Path WSMan:\localhost\Service\AllowUnencrypted -Value True

Set-Item -Path WSMan:\localhost\Service\Auth\Basic -Value True

Enable-NetFirewallRule -Name "WINRM-HTTP-In-TCP"
```

- Konfiguration vom vars File für http auf dem Linux Server

```
ansible_user="<user>"
ansible_password="<Passwort>"
ansible_connection=winrm
ansible_port=5985
ansible_winrm_server_cert_validation=ignore
ansible_winrm_scheme=http
```

Über HTTPS mit Self-Signed-Cert verbinden (verschlüsselt)

- Auf den Windows hosts Zertifikat erstellen:

```
New-SelfSignedCertificate -DnsName "WS1-Ansible" -CertStoreLocation
Cert:\LocalMachine\My
```

```
PS C:\Windows\system32> New-SelfSignedCertificate -DnsName "WS2-Ansible" -CertStoreLocation Cert:\LocalMachine\My

PSParentPath: Microsoft.PowerShell.Security\Certificate::LocalMachine\My

Thumbprint                               Subject
-----
FA8A0E1225C0F3B158EFDBD92EA7D195635FF8C8  CN=WS2-Ansible
```

- Der Thumbprint kann nochmal mit `Get-ChildItem Cert:\LocalMachine\My` angezeigt werden
- HTTPS Listener konfigurieren (erstellten thumbprint angeben):

```
winrm create winrm/config/Listener?Address=*&Transport=HTTPS
'@{CertificateThumbprint="BC67D5ADF5BD19E45A0C7FA70F28380A6A287652"}'
```

- HTTPS eingehend erlauben und authentication per winrm erlauben

```
netsh advfirewall firewall add rule name="WinRM HTTPS" dir=in action=allow
protocol=TCP localport=5986
```

```
winrm set winrm/config/service/Auth '@{Basic="true"}'
```

- Konfiguration vom vars File für https auf dem Linux Server

```
ansible_user="<user>"
ansible_password="<passwort>"
ansible_connection=winrm
ansible_port=5986
ansible_winrm_server_cert_validation=ignore
ansible_winrm_scheme=https
```

Konfiguration schreiben

Ordnerstruktur:

- inventories/
 - hosts
 - group_vars/
 - windows_nodes/
 - windows_nodes.yaml
 - vault.yaml
- roles/
 - windows_role/
 - tasks/
 - main.yaml
 - install_chocolatey.yaml
 - install_7zip.yaml
 - win_updates.yaml
 - win_hotfix.yaml
 - local.yaml
 - domain.yaml
 - win_commands.yaml
- playbooks/
 - windows_config

ansible.cfg

hosts file

- Beinhaltet die Adressen der Server

```
[windows_nodes]
ws1_ansible ansible_host=192.168.1.2
ws2_ansible ansible_host=192.168.1.3
```

windows_nodes file

- Setzt die Variablen die zur Verbindung benötigt werden

```
ansible_connection: winrm
ansible_port: 5986
ansible_winrm_server_cert_validation: ignore
ansible_winrm_scheme: https
```

vault file

- Es wird ein Vault file für das verschlüsselte Speichern von Benutzernamen und Passwörtern der Server

```
# Erstellen (Passwort angeben)
ansible-vault create vault.yaml

# Bearbeiten
ansible-vault edit vault.yaml

# Inhalt ausgeben
ansible-vault view vault.yaml
```

Inhalt:

```
ansible_user: "domain\\<username>"
ansible_password: "password"
```

install_chocolatey file

```
- name: Installiert Chocolatey
win_chocolatey:
  name: chocolatey
  state: present
```

install_7zip file

```
- name: Installiert 7-Zip über Chocolatey
  win_chocolatey:
    name: 7zip
    state: present
```

win_updates file

```
- name: Install all critical and security updates
  win_updates:
    category_names:
      - CriticalUpdates
      - SecurityUpdates
    state: installed
    register: update_result

- name: Reboot host if required
  win_reboot:
    when: update_result.reboot_required
```

win_hotfix

```
- name: Download KB3172729 for Server 2012 R2
  win_get_url:
    url:
      http://download.windowsupdate.com/d/msdownload/update/software/secu/2016/07/windows8.1-kb3172729-x64_e8003822a7ef4705cbb65623b72fd3cec73fe222.msu
    dest: C:\temp\KB3172729.msu

- name: Install hotfix
  win_hotfix:
    hotfix_kb: KB3172729
    source: C:\temp\KB3172729.msu
    state: present
    register: hotfix_result

- name: Reboot host if required
  win_reboot:
    when: hotfix_result.reboot_required
```

local

```
- name: Create local group to contain new users
win_group:
  name: LocalGroup
  description: Allow access to C:\Development folder

- name: Create local users
win_user:
  name: "{{ item.name }}"
  password: "{{ item.password }}"
  groups: "LocalGroup"
  update_password: on_create
  password_never_expires: true
loop:
  - { name: "User1", password: "Password1" }
  - { name: "User2", password: "Password2" }

- name: Create Development folder
win_file:
  path: C:\Development
  state: directory

- name: Set ACL of Development folder
win_acl:
  path: C:\Development
  rights: FullControl
  state: present
  type: allow
  user: LocalGroup

- name: Remove parent inheritance of Development folder
win_acl_inheritance:
  path: C:\Development
  reorganize: true
  state: absent
```

domain

```
- name: Create global group
win_domain_group:
  name: 'G_test'
  scope: 'global'
  state: present
  domain: corp.at
  delegate_to: ws1_ansible
  run_once: true

- name: Create domain local group and add global group as member
win_domain_group:
  name: 'DL_test'
  scope: 'domainlocal'
```

```
members:
- 'CORP\G_test'
state: present
domain: corp.at
delegate_to: ws1_ansible
run_once: true

- name: Create domain user and add to global group
win_domain_user:
  name: user
  password: "Ganzgeheim123!"
  update_password: always
  state: present
  groups:
  - 'G_test'
delegate_to: ws1_ansible
run_once: true
```

win_commands

```
- name: Ensure folder test
win_shell: if not exist C:\Users\janwi\Desktop\test mkdir
C:\Users\janwi\Desktop\test
args:
  executable: cmd.exe

- name: Run an executable using win_command
win_command: whoami.exe
```

main.yaml file

- Man könnte auch alle tasks direkt in dieses File schreiben, Übersicht wäre aber dadurch schlechter

```
- import_tasks: install_chocolatey.yaml
- import_tasks: install_7zip.yaml
```

Befehl: ansible-playbook

-i inventories/hosts

playbooks/windows_config.yml

--ask-vault-pass